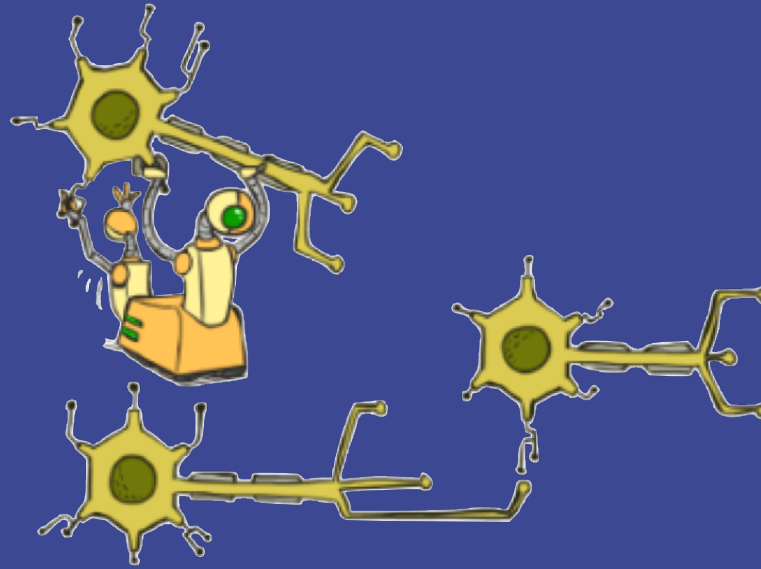


CS 156

Intro to AI

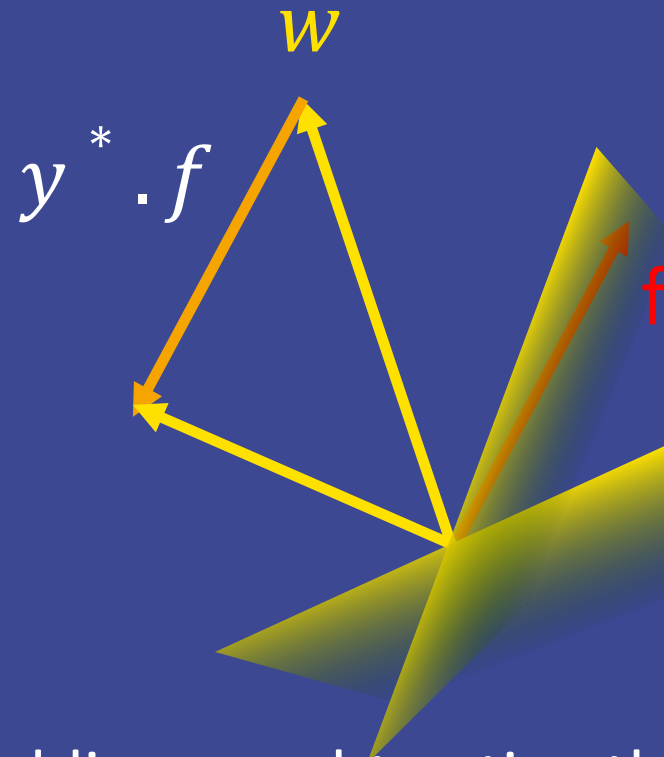


From Perceptrons to Neural Nets

These slides are based on the slides created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.
The artwork is by Ketrina Yim

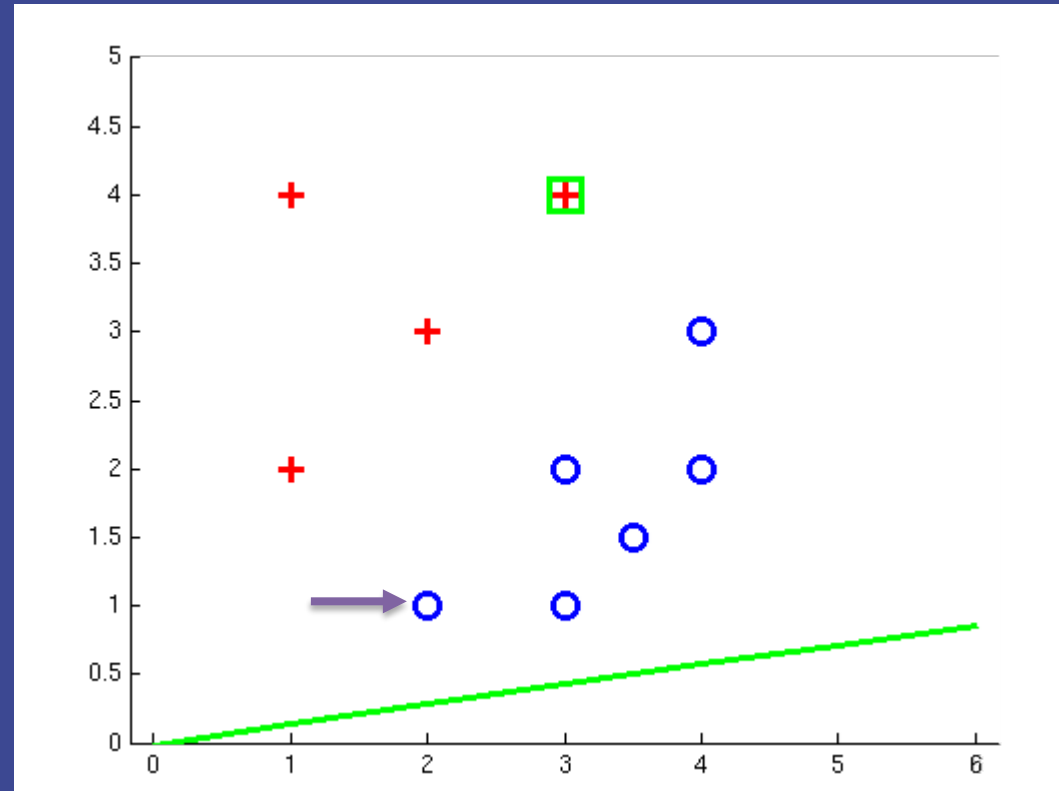
Learning: Binary Perceptron

- ▶ Start with weights = 0
- ▶ For each training instance:
 - Classify with current weights
 - $y = \begin{cases} \text{Positive class if } w \cdot f(x) > 0 \\ \text{Other class if } w \cdot f(x) \leq 0 \end{cases}$
 - y^* : training label
 - If correct (i.e., $y=y^*$), no change!
 - If wrong: adjust the weight vector by adding or subtracting the feature vector: $w = w \pm f$ (subtract if y^* is the negative class)
- ▶ Repeat until no more significant changes (iterations)



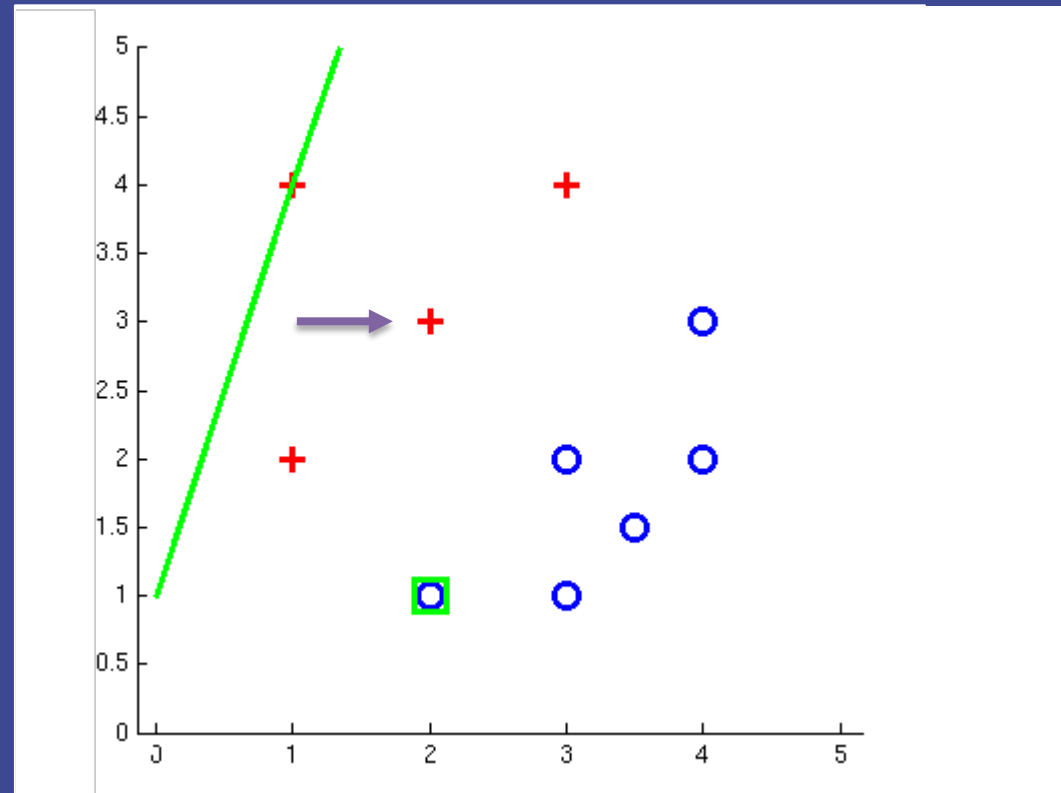
Example: Perceptron

► Separable Case



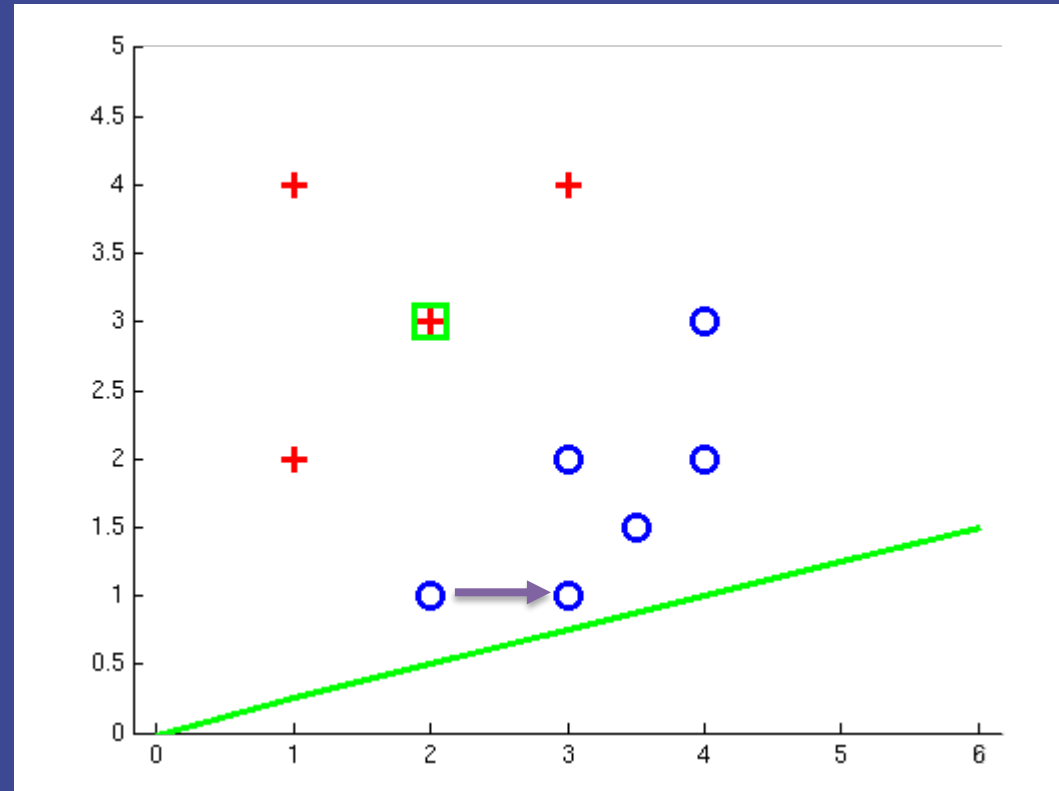
Example: Perceptron

► Separable Case



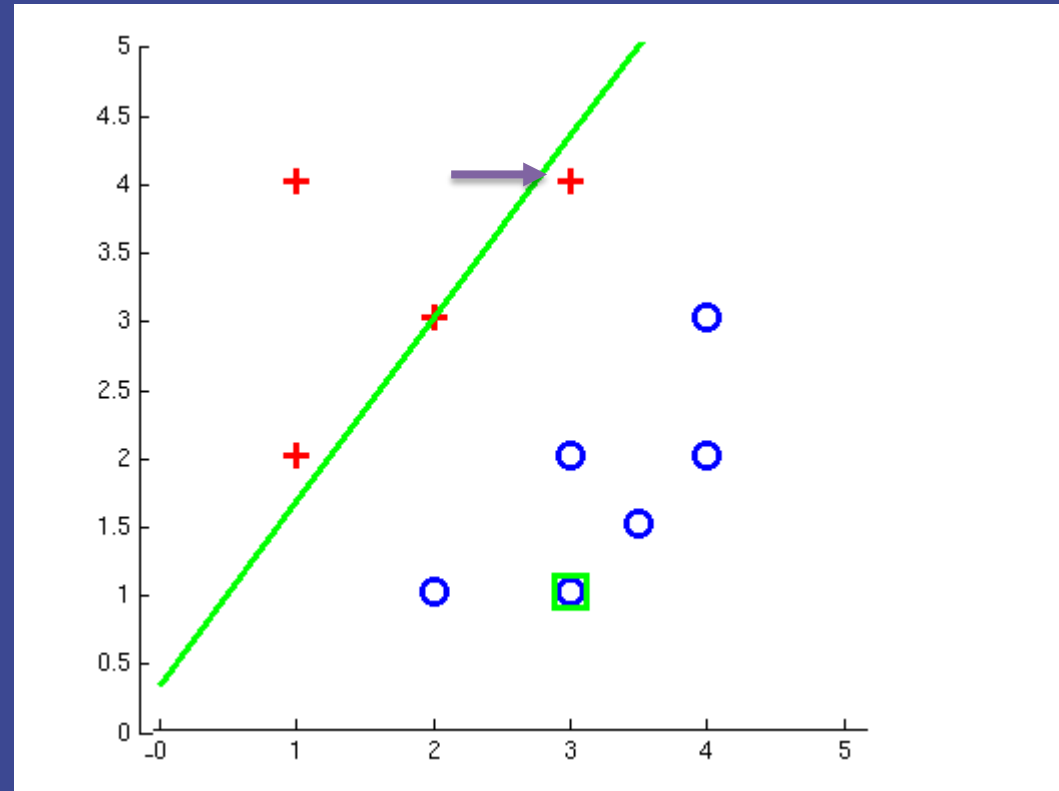
Example: Perceptron

► Separable Case



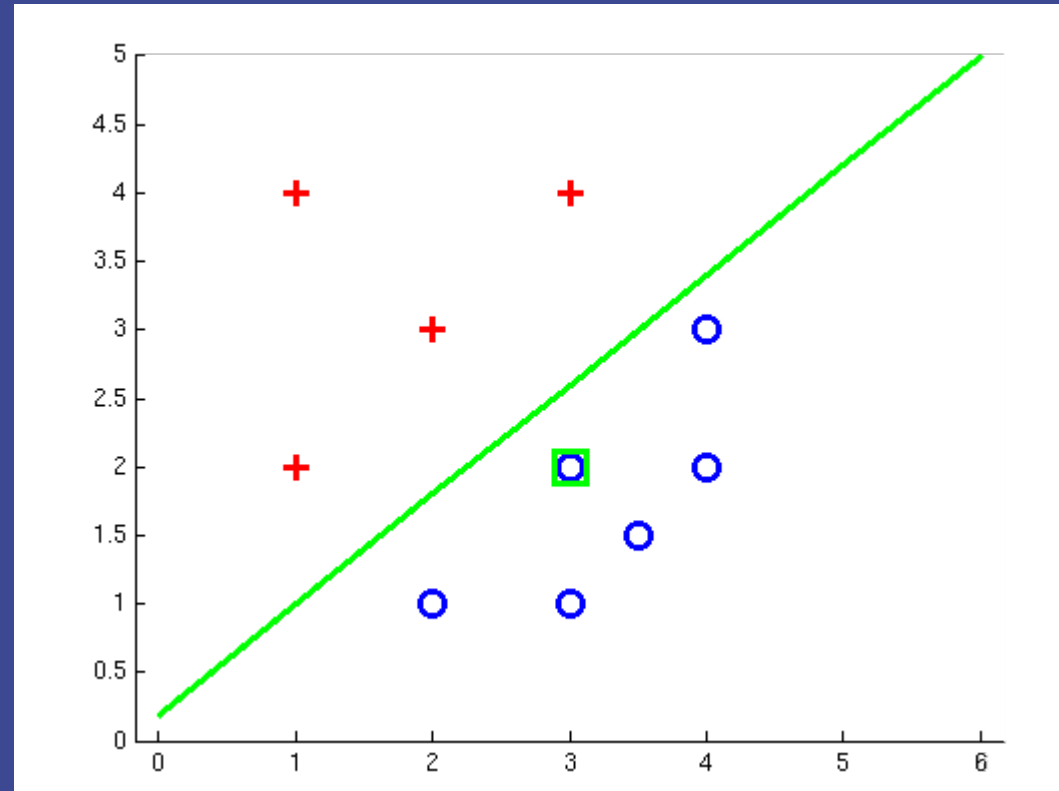
Examples: Perceptron

- Separable Case



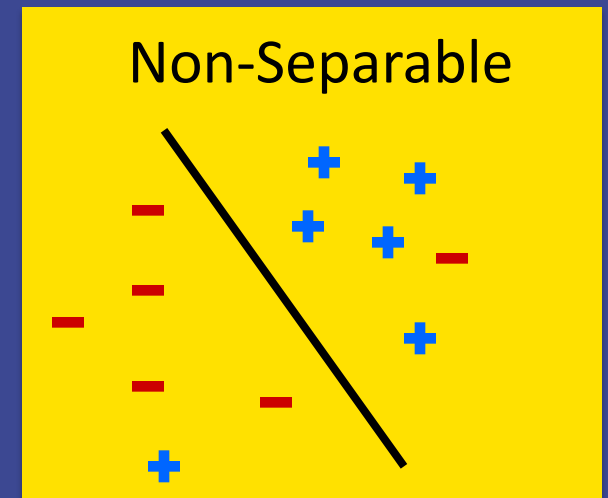
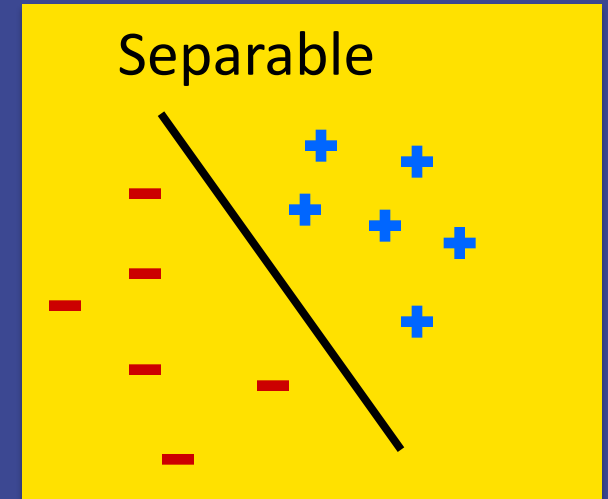
Examples: Perceptron

► Separable Case



Properties of Perceptrons

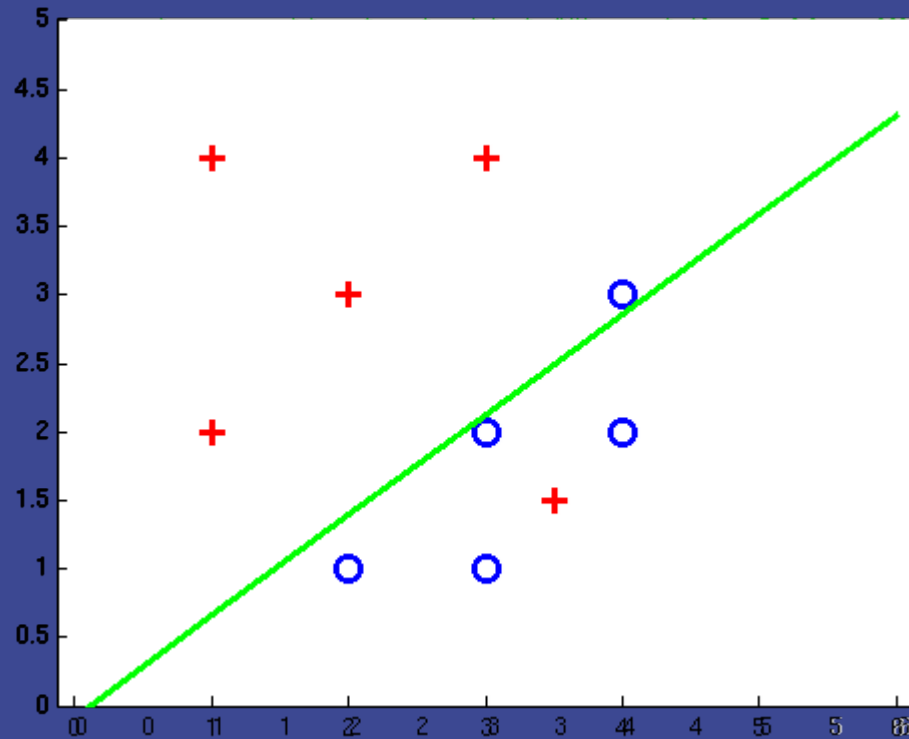
- ▶ **Separability**: some parameters get the training set perfectly correct
- ▶ Not all data is separable
- ▶ **Convergence**: if the training data is separable, the binary perceptron rule is guaranteed to converge



Examples: Perceptron

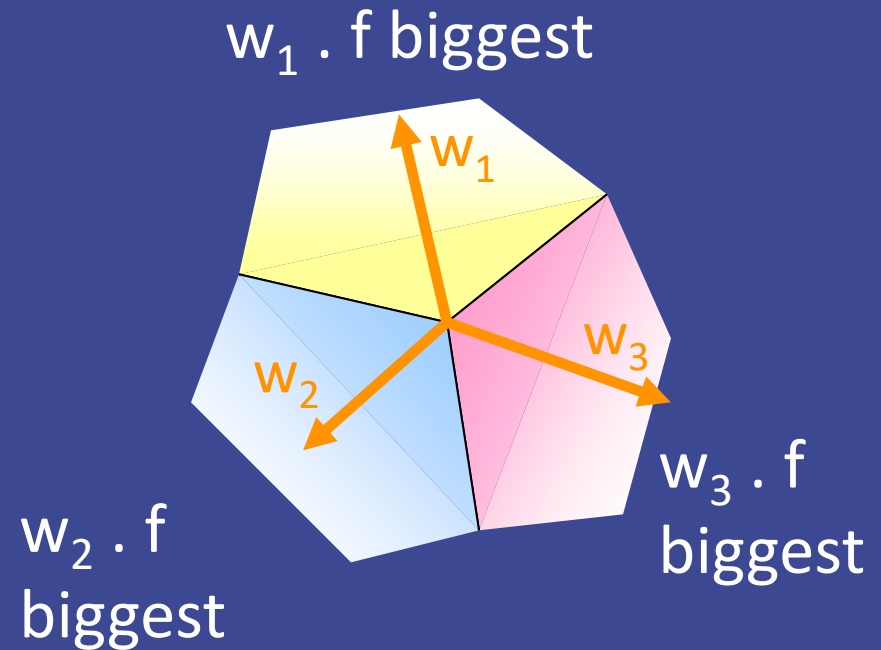
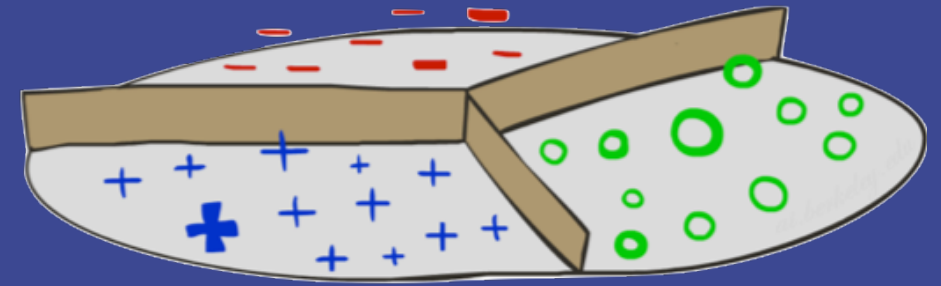
► Non-Separable Case

- average



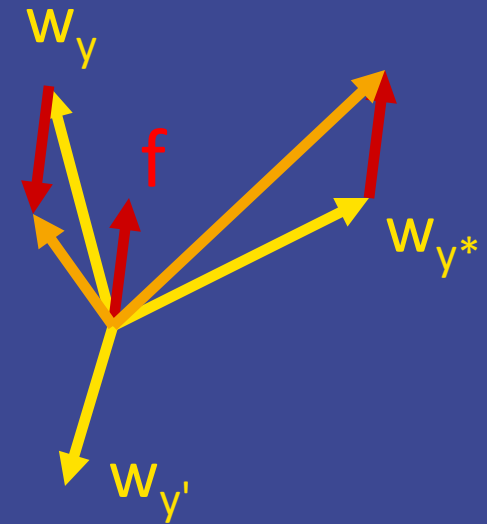
Multiclass Decision Rule

- ▶ If we have multiple classes:
 - A weight vector for each class: w_y
 - Score (activation) of a class y :
 $w_y \cdot f(x)$
 - Prediction: highest score wins:
 $y = \operatorname{argmax}_y w_y \cdot f(x)$
 $y = \text{label corresponding to the highest score: } w_y \cdot f(x)$



Learning: Multiclass Perceptron

- ▶ Start with all weights = 0
- ▶ Pick up training examples one by one
- ▶ Predict with current weights
$$y = \operatorname{argmax}_y w_y \cdot f(x)$$
- ▶ If correct ($y = y^*$), no change!
- ▶ If wrong: lower score of wrong answer (y), raise score of right answer (y^*)
$$w_y = w_y - f(x)$$
$$w_{y^*} = w_{y^*} + f(x)$$
- ▶ Repeat until no more significant changes (iterations)



Example: Multiclass Perceptron

Training Examples

"win the vote"

"win the election"

"win the game"

Training Labels

POLITICS

POLITICS

SPORTS

Possible Labels:

- POLITICS
- SPORTS
- TECH

Example: Multiclass Perceptron

$f(\text{"win the vote"})$

What is $f(\text{"win the vote"})$?

BIAS : ?
win : ?
game : ?
vote : ?
the : ?

Example: Multiclass Perceptron

$f(\text{"win the vote"})$

What is $f(\text{"win the vote"})$?

A

BIAS : 0
win : 0
game : 0
vote : 0
the : 0

B

BIAS : 1
win : 0
game : 0
vote : 0
the : 0

C

BIAS : 0
win : 1
game : 0
vote : 1
the : 1

D

BIAS : 1
win : 1
game : 0
vote : 1
the : 1

E

BIAS : 1
win : 1
game : 1
vote : 1
the : 1

Example: Multiclass Perceptron

Training Examples

"win the vote"

"win the election"

"win the game"

W_{SPORTS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Training Labels

POLITICS

POLITICS

SPORTS

W_{POLITICS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

$f(\text{"win the vote"})$

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

W_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the vote"})$

$y = ?$

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

$w_{\text{SPORTS}} \cdot f(\text{"win the vote"}) =$

What is $w_{\text{SPORTS}} \cdot f(\text{"win the vote"})$?

w_{SPORTS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

w_{POLITICS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the vote"})$

$y = ?$

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the vote"}) = 0$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the vote"}) = 0$$

$$w_{\text{TECH}} \cdot f(\text{"win the vote"}) = 0$$

w_{SPORTS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

w_{POLITICS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the vote"})$

$y = \text{SPORTS}$

$y^* = \text{POLITICS}$

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the vote"}) = 0$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the vote"}) = 0$$

$$w_{\text{TECH}} \cdot f(\text{"win the vote"}) = 0$$

w_{SPORTS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

w_{POLITICS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

We are supposed to pick the class with the highest value of the dot product. Since the 3 products are equal, we pick SPORTS arbitrarily.

Example: Multiclass Perceptron

f ("win the vote")

$y = \text{SPORTS}$

$y^* = \text{POLITICS}$

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

$$w_{\text{SPORTS}} = w_{\text{SPORTS}} - f$$

$$w_{\text{POLITICS}} = w_{\text{POLITICS}} + f$$

w_{SPORTS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

W_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

W_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

W_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the election"})$

BIAS	:	1
win	:	1
game	:	0
vote	:	0
the	:	1

$w_{\text{SPORTS}} \cdot f(\text{"win the election"}) =$

What is $w_{\text{SPORTS}} \cdot f(\text{"win the election"})$?

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the election"})$

$y = ?$

BIAS	:	1
win	:	1
game	:	0
vote	:	0
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the election"}) = -3$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the election"}) = 3$$

$$w_{\text{TECH}} \cdot f(\text{"win the election"}) = 0$$

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the election"})$

$y = \text{POLITICS}$

$y^* = \text{POLITICS}$

BIAS	:	1
win	:	1
game	:	0
vote	:	0
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the election"}) = -3$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the election"}) = 3$$

$$w_{\text{TECH}} \cdot f(\text{"win the election"}) = 0$$

No updates

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the game"})$

BIAS	:	1
win	:	1
game	:	1
vote	:	0
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the game"}) = -3$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the game"}) = 3$$

$$w_{\text{TECH}} \cdot f(\text{"win the game"}) = 0$$

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the game"})$

$y = ?$

- A. SPORTS
- B. POLITICS
- C. TECH

BIAS	:	1
win	:	1
game	:	1
vote	:	0
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the game"}) = -3$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the game"}) = 3$$

$$w_{\text{TECH}} \cdot f(\text{"win the game"}) = 0$$

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the game"})$

$y = \text{POLITICS}$

$y^* = \text{SPORTS}$

BIAS	:	1
win	:	1
game	:	1
vote	:	0
the	:	1

$$w_{\text{SPORTS}} \cdot f(\text{"win the game"}) = -3$$

$$w_{\text{POLITICS}} \cdot f(\text{"win the game"}) = 3$$

$$w_{\text{TECH}} \cdot f(\text{"win the game"}) = 0$$

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

$f(\text{"win the game"})$

$y = \text{POLITICS}$

$y^* = \text{SPORTS}$

BIAS	:	1
win	:	1
game	:	1
vote	:	0
the	:	1

$$w_{\text{POLITICS}} = w_{\text{POLITICS}} - f$$

$$w_{\text{SPORTS}} = w_{\text{SPORTS}} + f$$

w_{SPORTS}

BIAS	:	-1
win	:	-1
game	:	0
vote	:	-1
the	:	-1

BIAS	:	0
win	:	0
game	:	1
vote	:	-1
the	:	0

w_{POLITICS}

BIAS	:	1
win	:	1
game	:	0
vote	:	1
the	:	1

BIAS	:	0
win	:	0
game	:	-1
vote	:	1
the	:	0

w_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

W_{SPORTS}

BIAS	:	0
win	:	0
game	:	1
vote	:	-1
the	:	0

W_{POLITICS}

BIAS	:	0
win	:	0
game	:	-1
vote	:	1
the	:	0

W_{TECH}

BIAS	:	0
win	:	0
game	:	0
vote	:	0
the	:	0

Example: Multiclass Perceptron

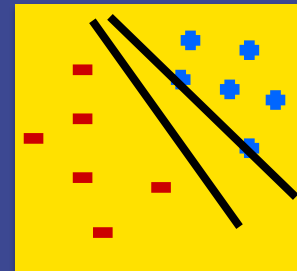
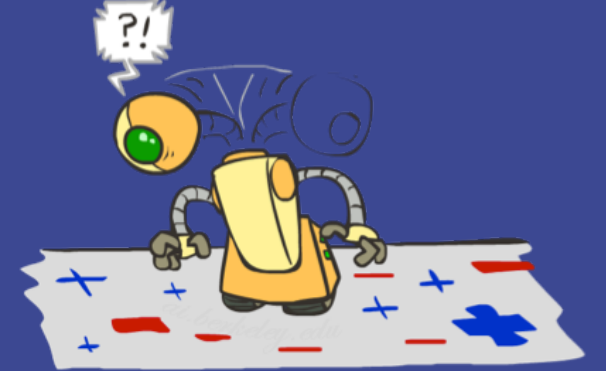
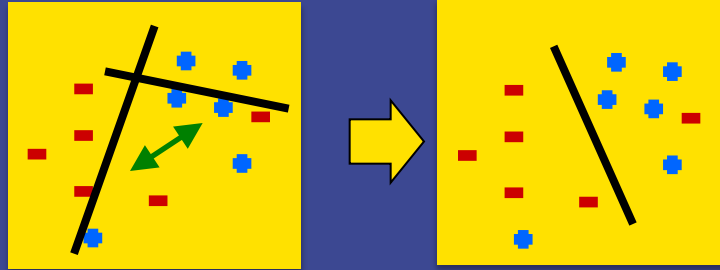
We're not done.

We go back for a second iteration over the examples and update the weights.

We keep iterating over the training examples until there are no more significant changes.

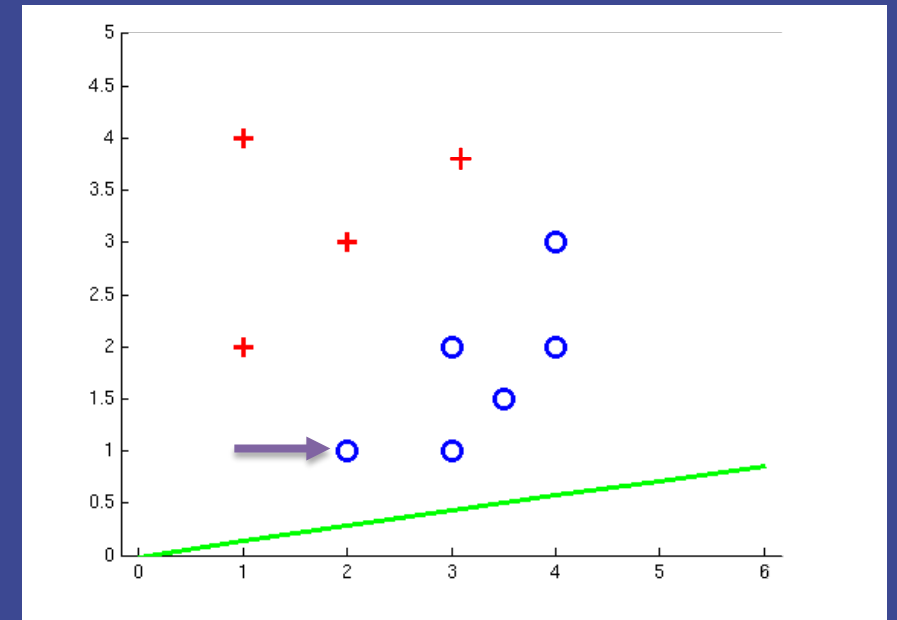
Problems with the Perceptron

- ▶ Noise: if the data isn't separable, weights might thrash.
 - averaging weight vectors over time can help (averaged perceptron)
- ▶ Mediocre generalization: finds a "barely" separating solution



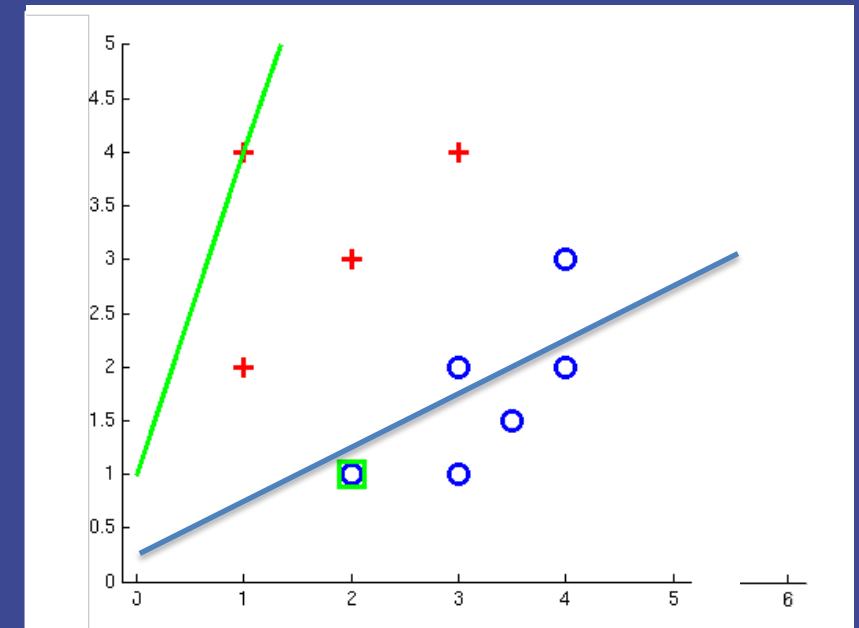
Improving the Perceptron

- ▶ Idea: adjust the weight update to mitigate the problems



Improving the Perceptron

- ▶ Idea: adjust the weight update to mitigate the problems
- ▶ MIRA (Margin Infused Relaxed Algorithm) : choose an update size that fixes the current mistake... but, minimizes the change to w



Minimum Correcting Update

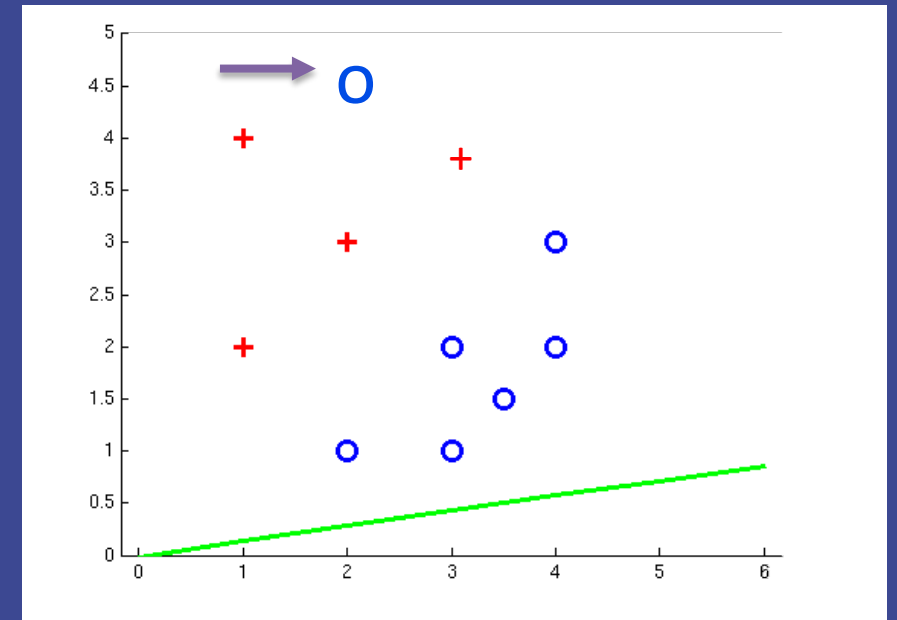
- ▶ Idea: adjust the weight update to mitigate the problems
- ▶ MIRA (Margin Infused Relaxed Algorithm) : choose an update size that fixes the current mistake... but, minimizes the change to w
- ▶ Introduce a learning rate:

$$w_y = w_y - r \cdot f(x)$$

$$w_{y^*} = w_{y^*} + r \cdot f(x)$$

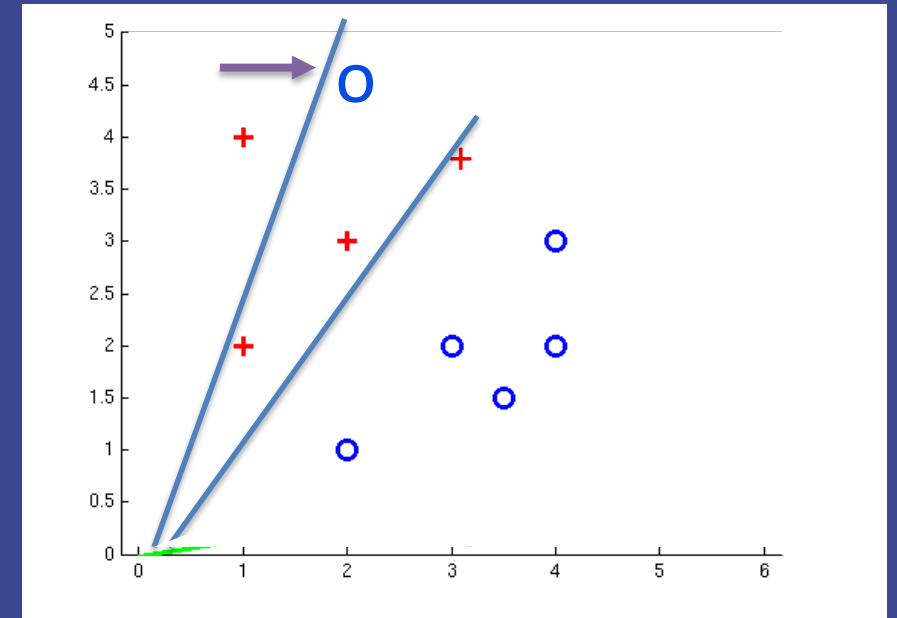
Maximum Step Size

- ▶ Example may be labeled incorrectly (noise)



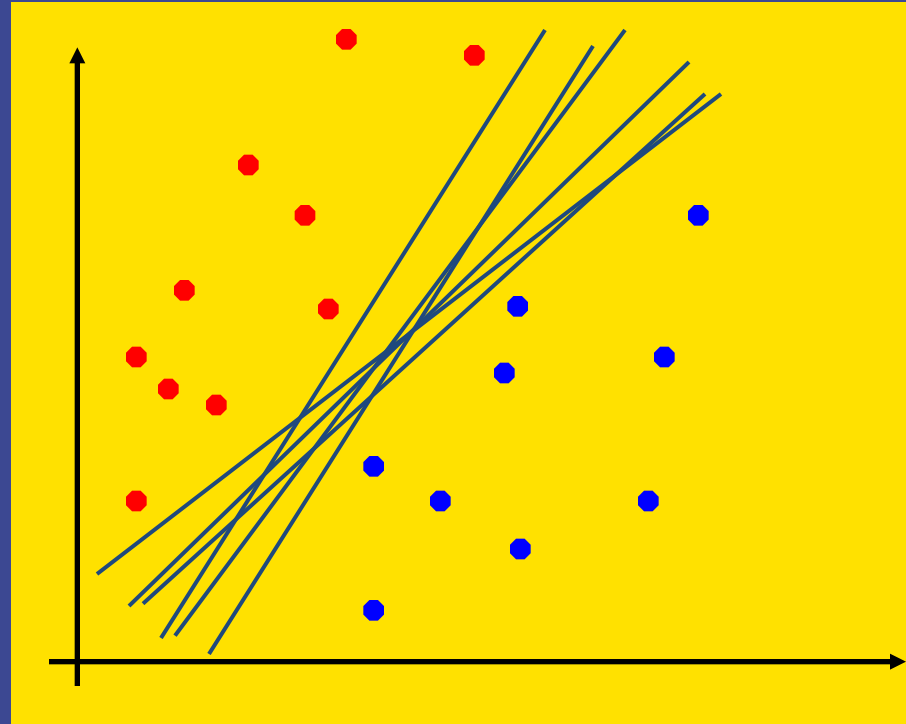
Maximum Step Size

- ▶ Example may be labeled incorrectly (noise)
- ▶ Solution: cap the maximum possible value of r with some constant
- ▶ Converges faster than perceptron
- ▶ Usually better, especially on noisy data



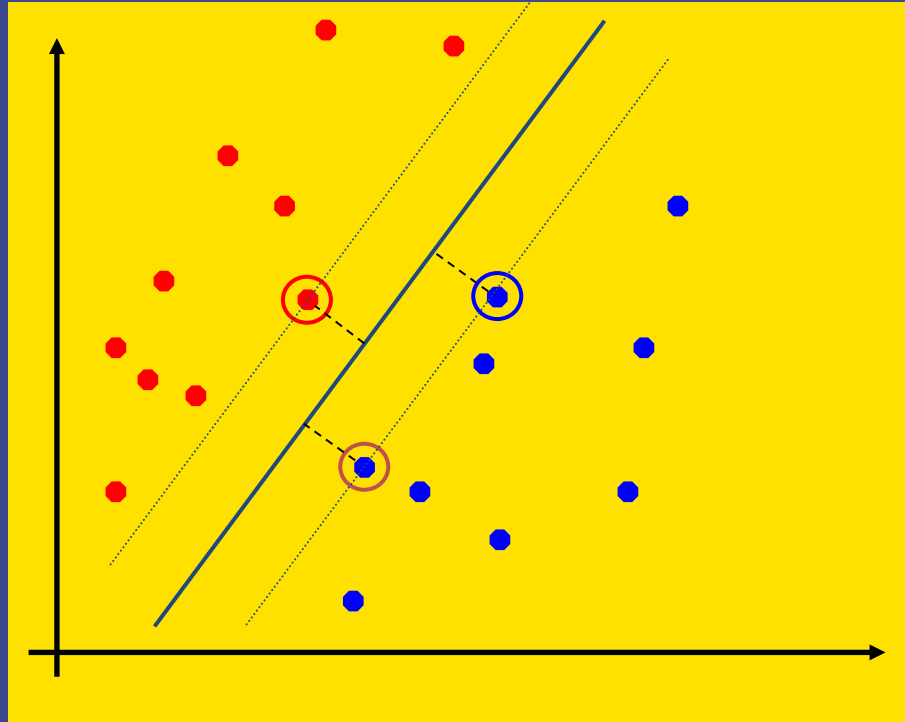
Linear Separators

- ▶ Which of these linear separators is better?



Support Vector Machines

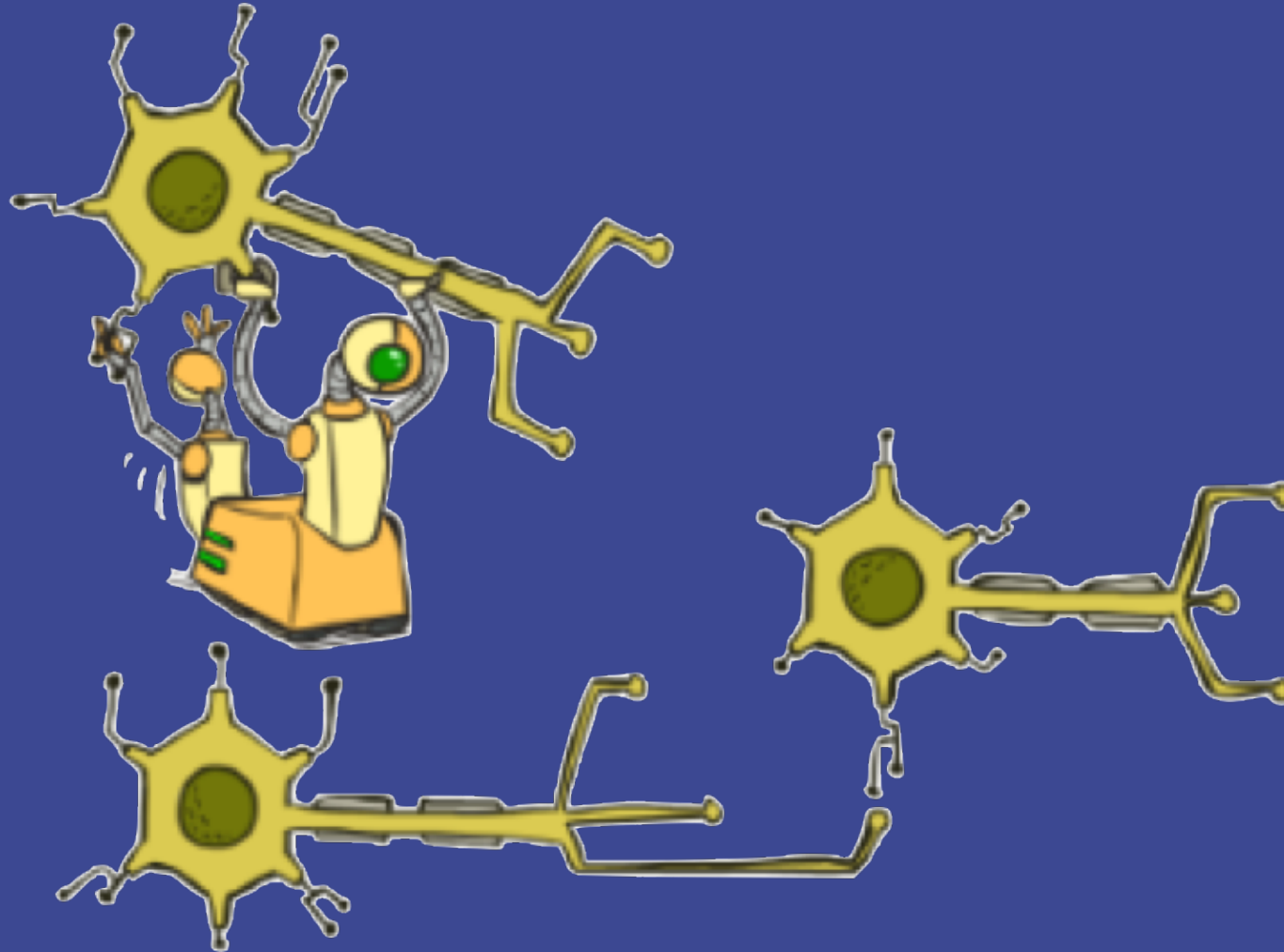
- ▶ **Maximizing the margin:** good according to intuition, theory, practice
- ▶ Only **support vectors** (vectors on the margin) matter; other training examples are ignored
- ▶ Support vector machines (SVMs) find the separator with the maximum margin



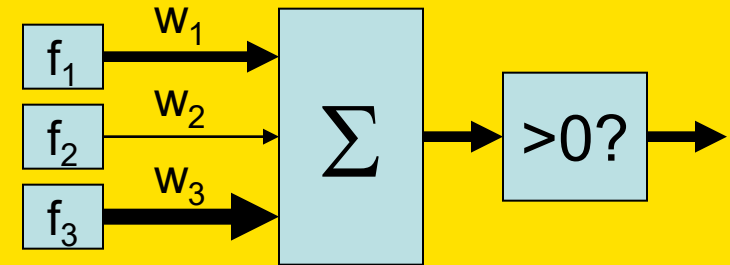
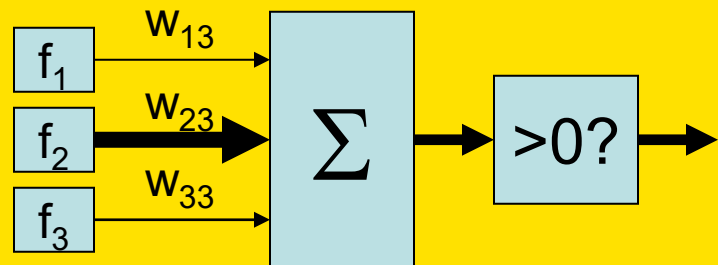
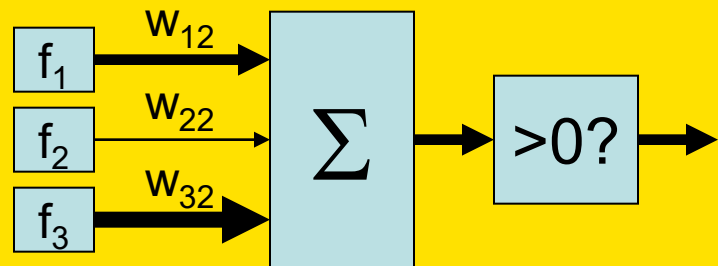
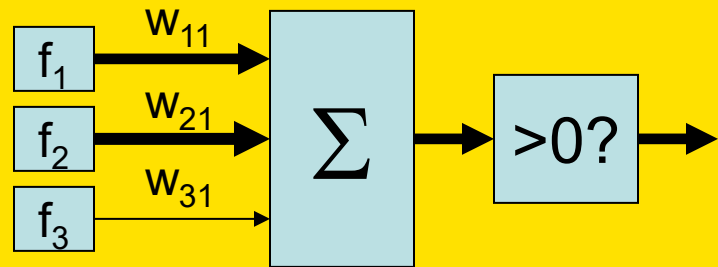
Classification: Comparison

- ▶ Naïve Bayes
 - Builds a model from training data
 - Gives prediction probabilities
 - Strong assumptions about feature independence
 - One pass through data (counting)
- ▶ Perceptron:
 - Makes fewer assumptions about data
 - Mistake-driven learning
 - Multiple passes through data
 - Often more accurate

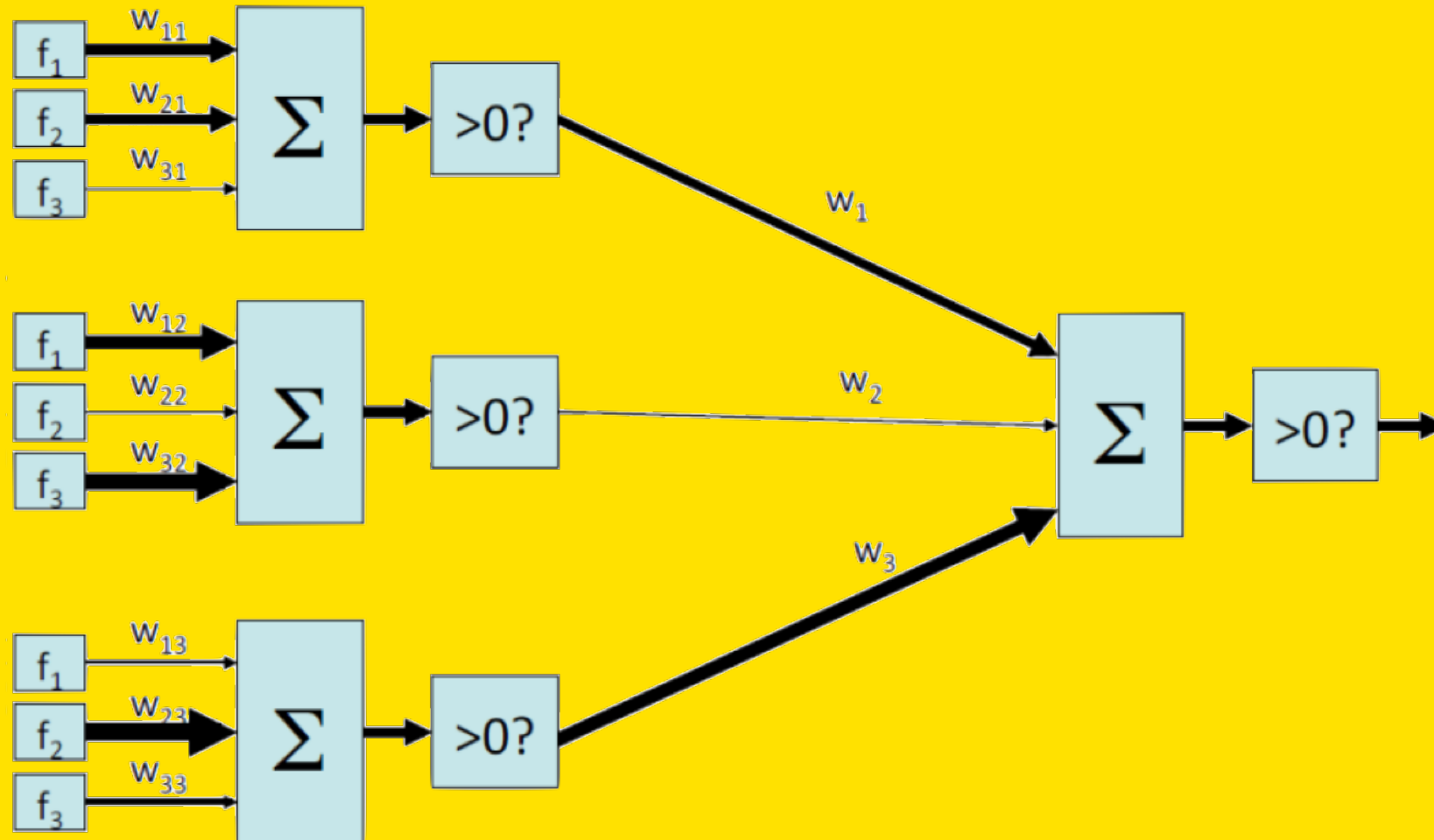
Artificial Neural Networks



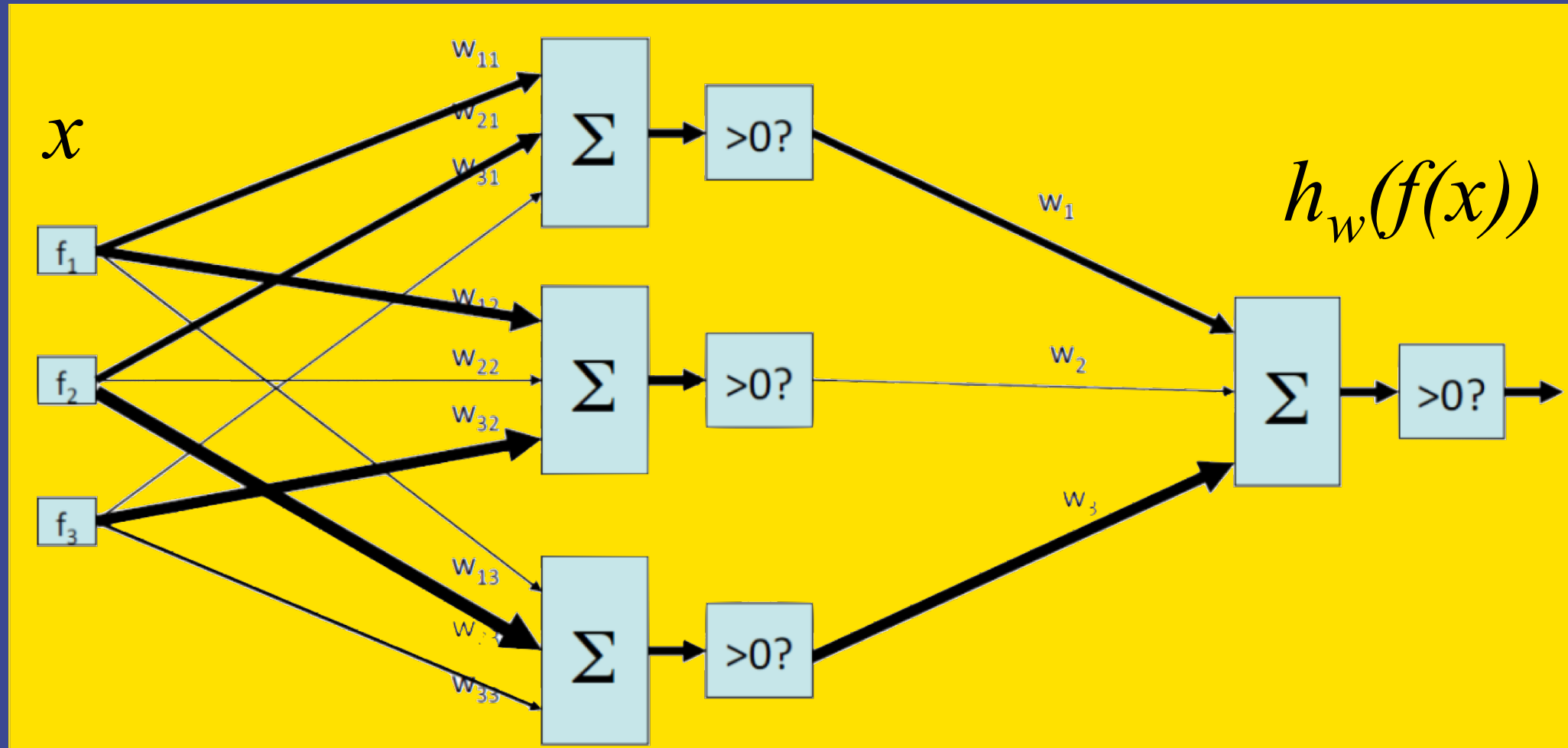
Two-Layer Perceptron Network



Two-Layer Perceptron Network



Two-Layer Perceptron Network



Learning w

- ▶ Training Examples:

$$(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(m)}, y^{(m)})$$

- ▶ Goal: find the weight vector w that **minimizes** the error (loss)

$$\min_w \sum_{i=1}^m (y^{(i)} - hw(f(x^{(i)})))^2$$

- ▶ How?

- Basic idea: hill climbing (gradient descent)

$w =$

$$\begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \\ w_{12} \\ w_{22} \\ w_{32} \\ w_{13} \\ w_{23} \\ w_{33} \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

Hill Climbing (gradient ascent/descent)

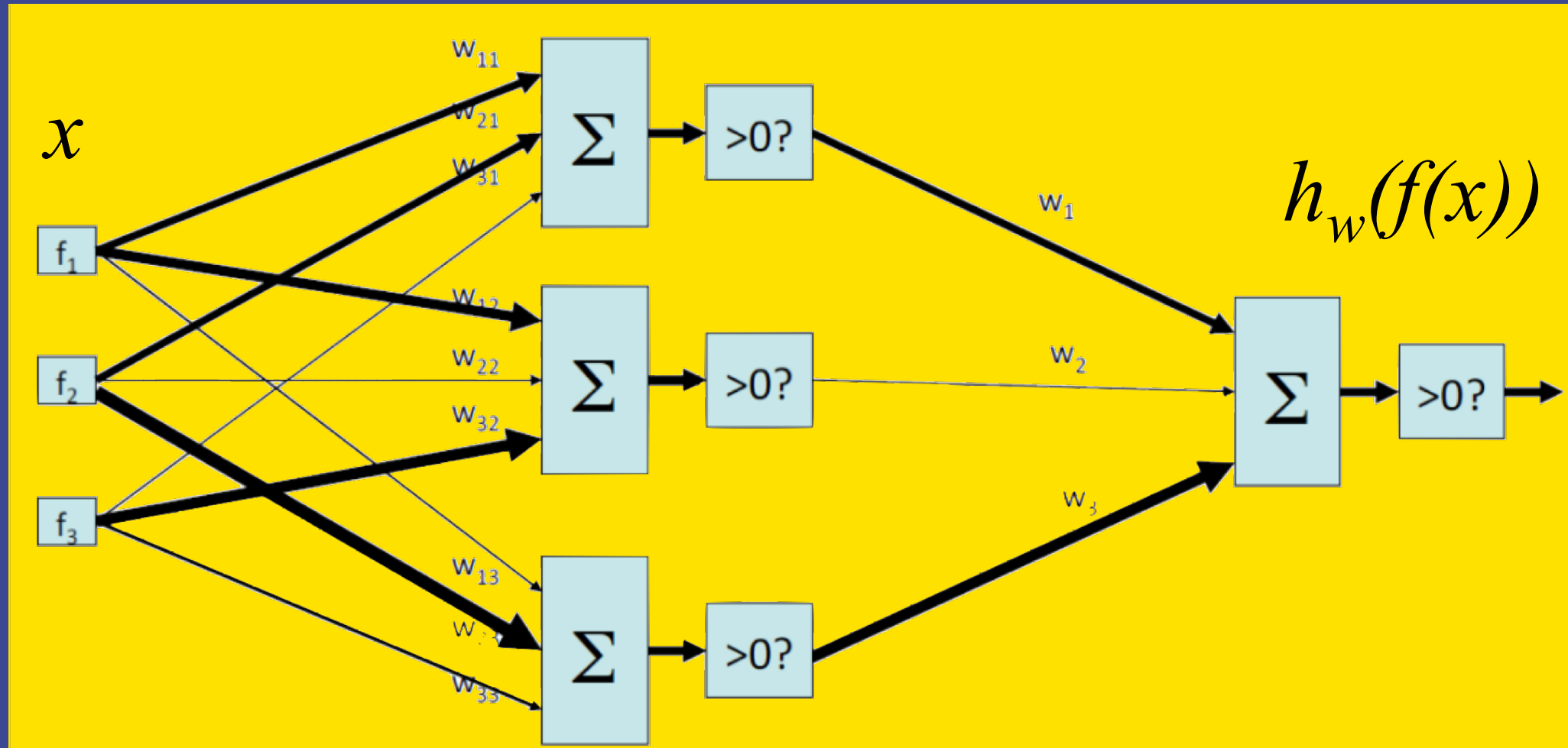
Simple, general idea:

- ▶ Start wherever
- ▶ Repeat: move to the best neighboring state
- ▶ If no neighbors better than current, quit
 - Neighbors = small perturbations of w

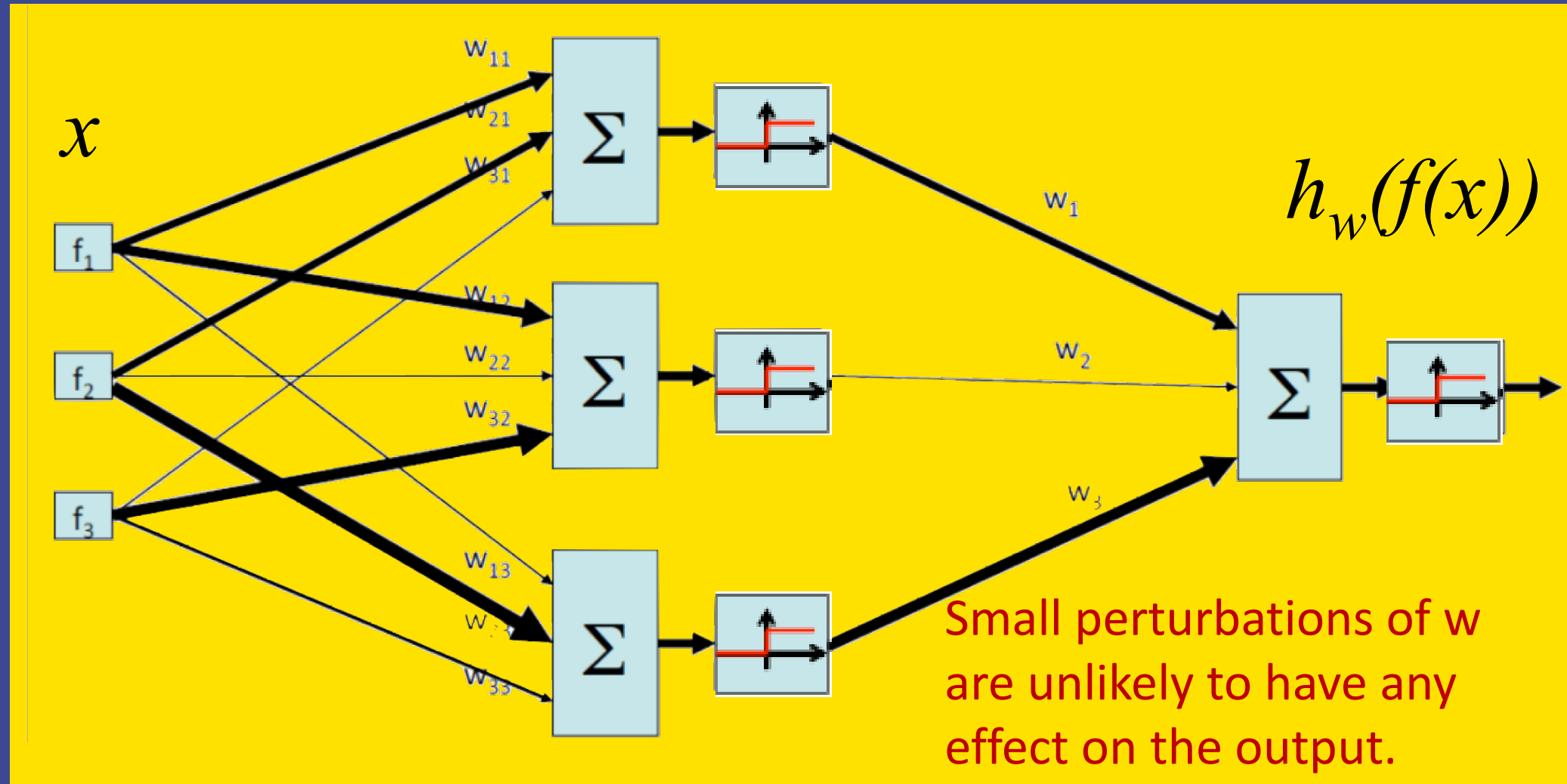
What's tricky when hill-climbing
for the multi-layer perceptron?



Two-Layer Perceptron Network

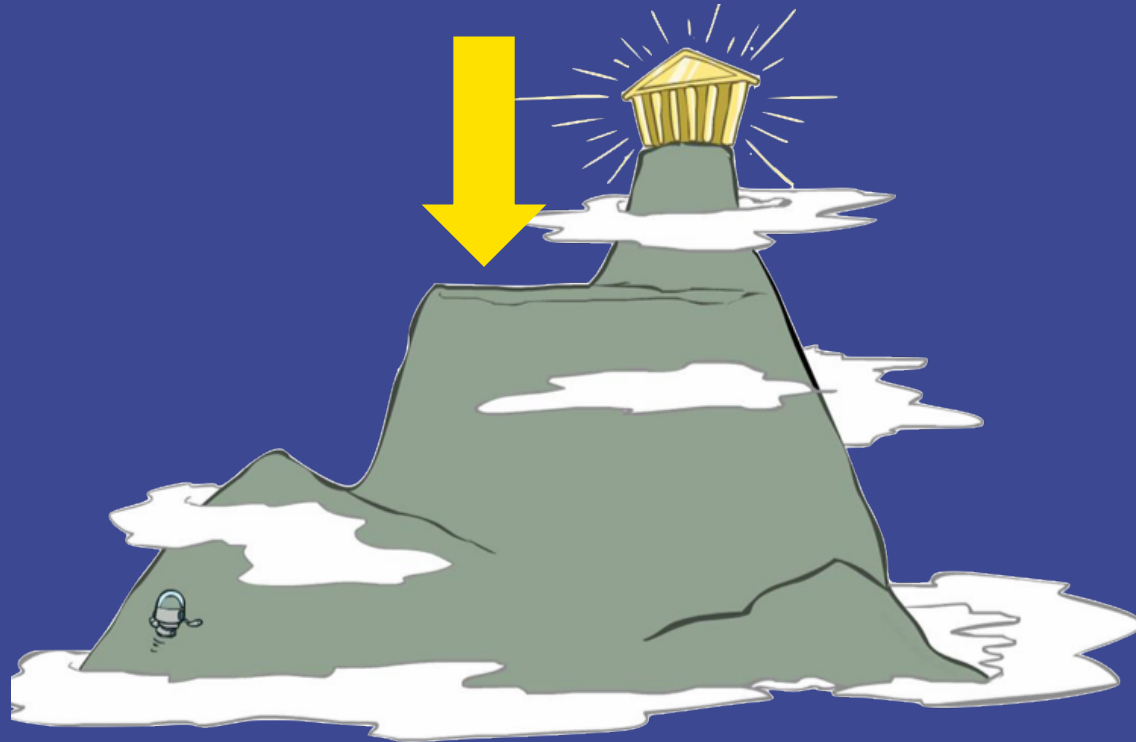


Two-Layer Perceptron Network

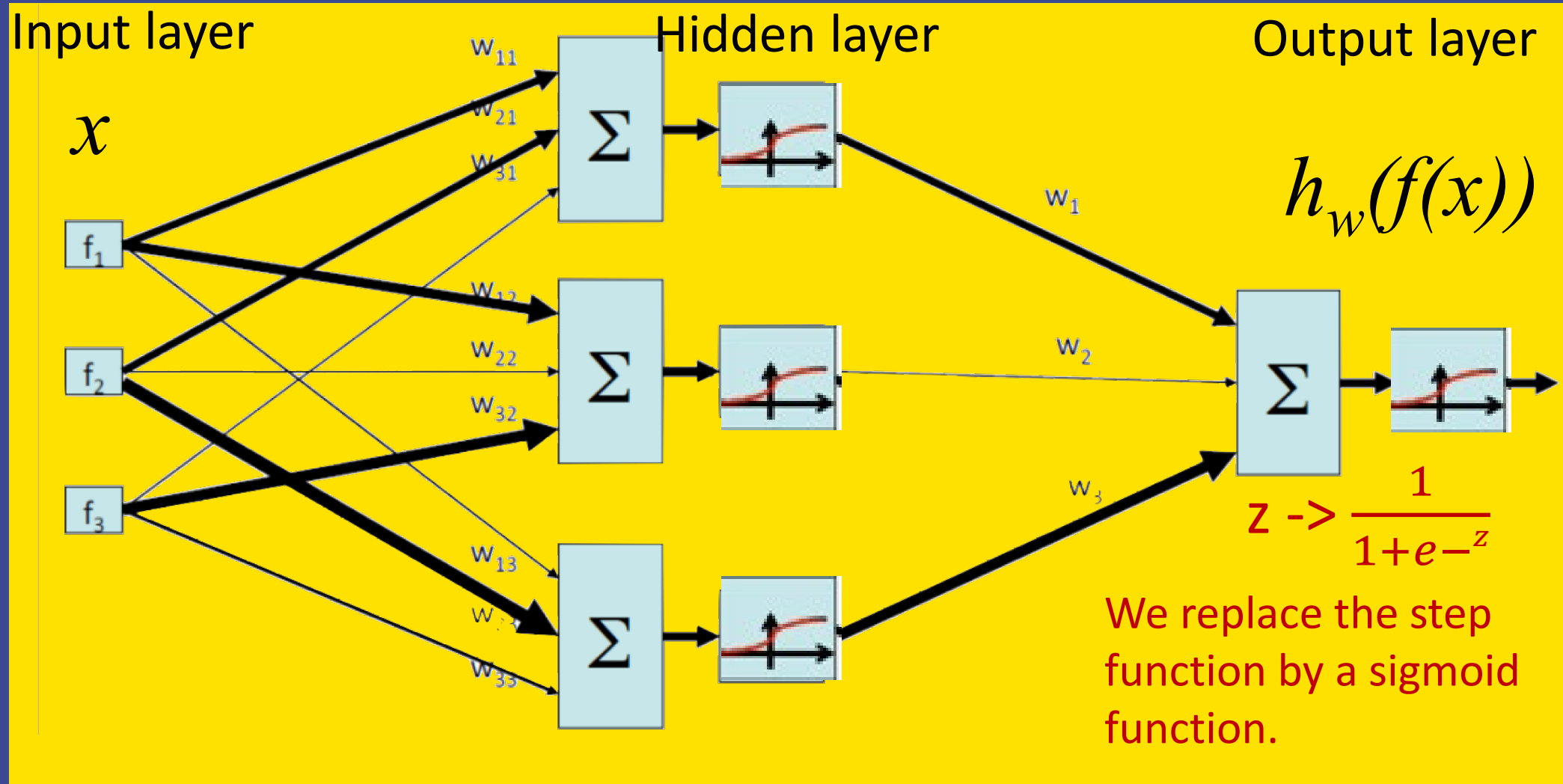


Two-Layer Perceptron Network

- ▶ Small perturbations of w are unlikely to have any effect on the output

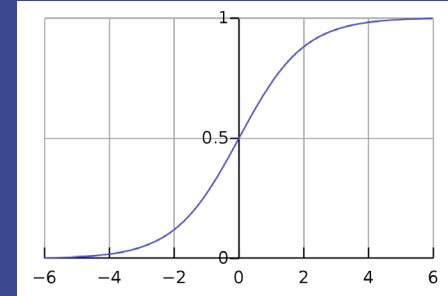


Sigmoid Neurons

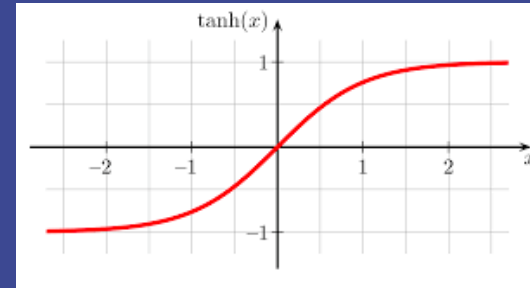


Activation Functions

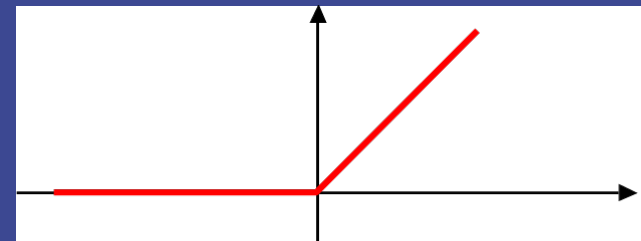
- ▶ Logistic Sigmoid function: $\frac{1}{1+e^{-z}}$



- ▶ Hyperbolic Tangent (tanh): $\frac{e^{2z} - 1}{e^{2z} + 1}$



- ▶ Rectified Linear Unit (ReLU): $\max(0, z)$

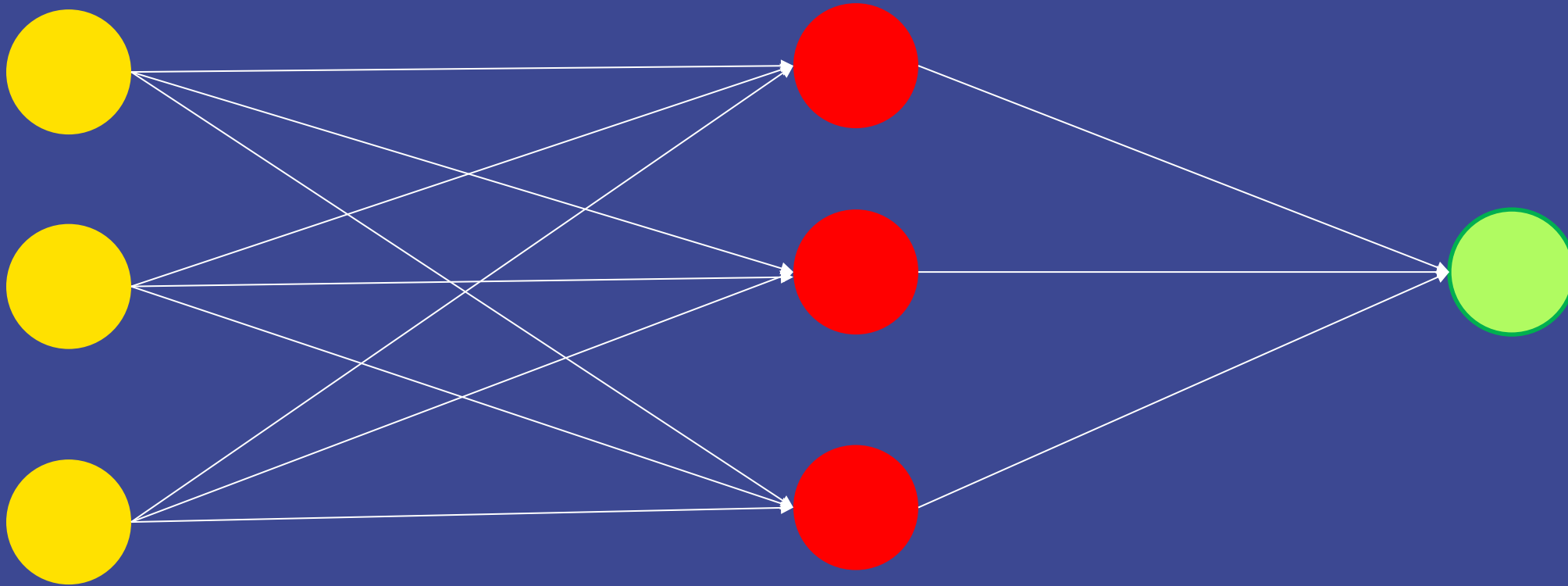


Two-Layer Neural Network

Input layer

Hidden layer

Output layer

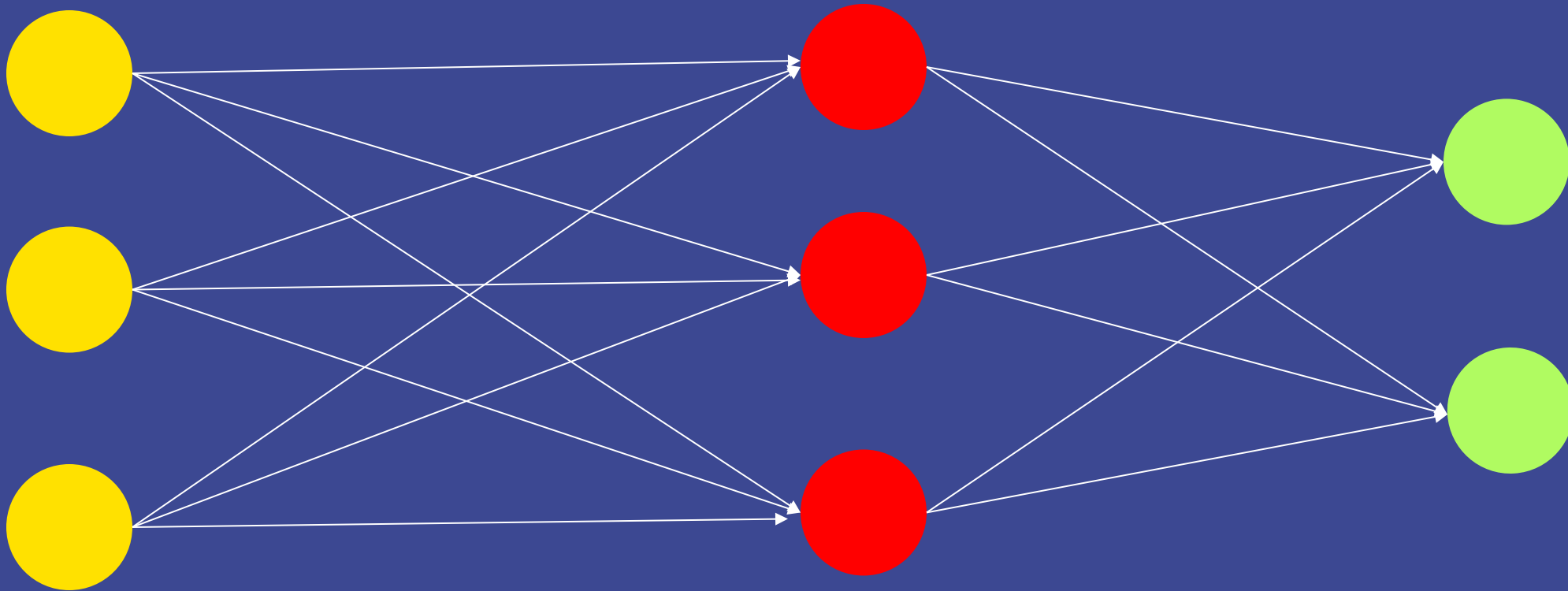


Two-Layer Neural Network

Input layer

Hidden layer

Output layer

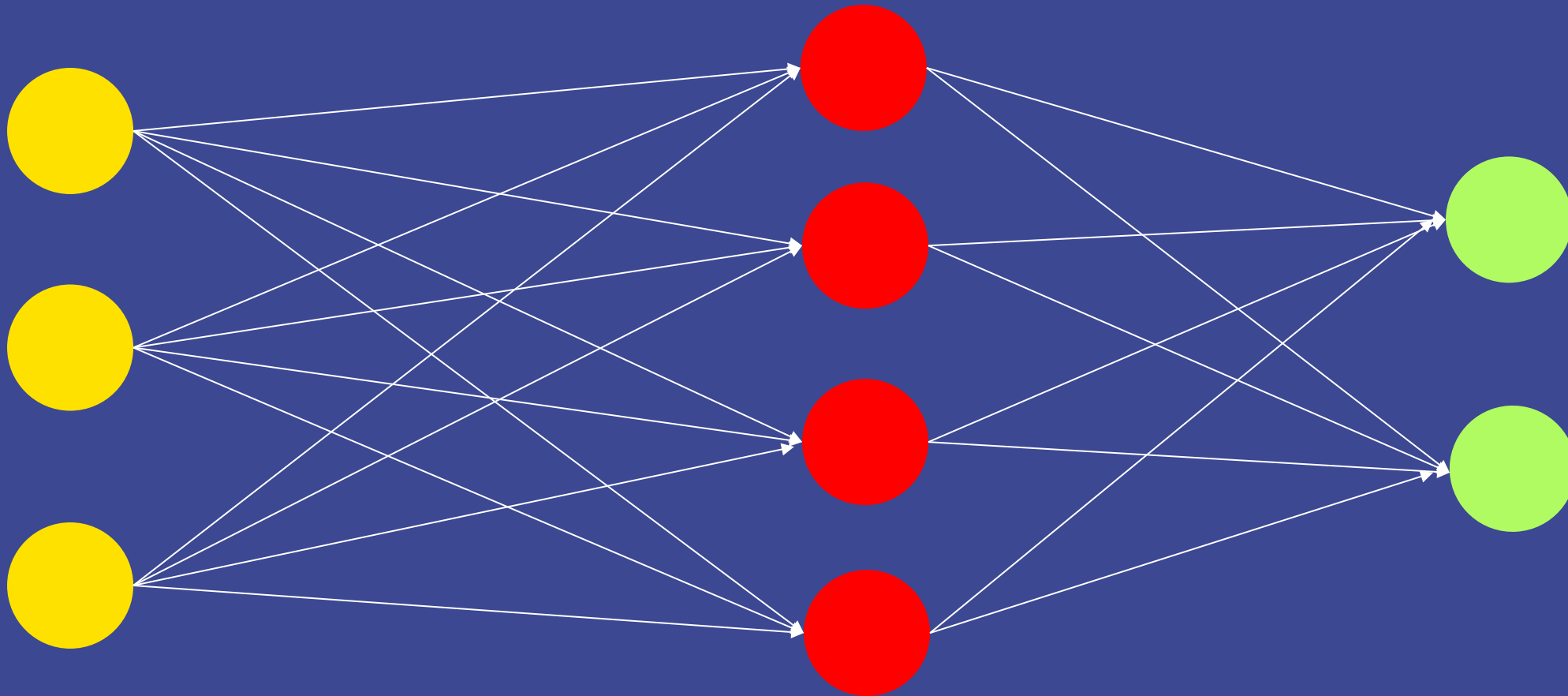


Two-Layer Neural Network

Input layer

Hidden layer

Output layer

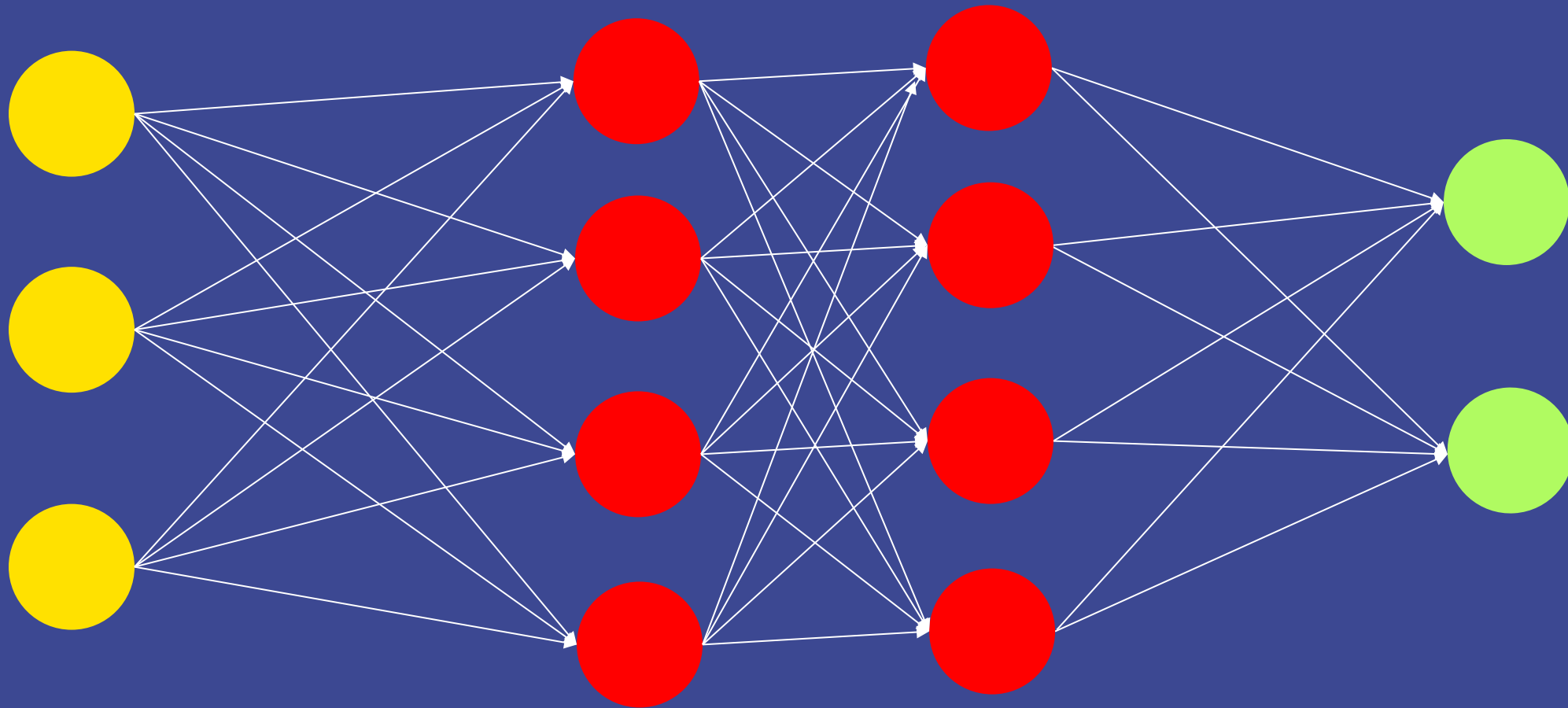


Deep Neural Network

Input layer

Hidden layers

Output layer

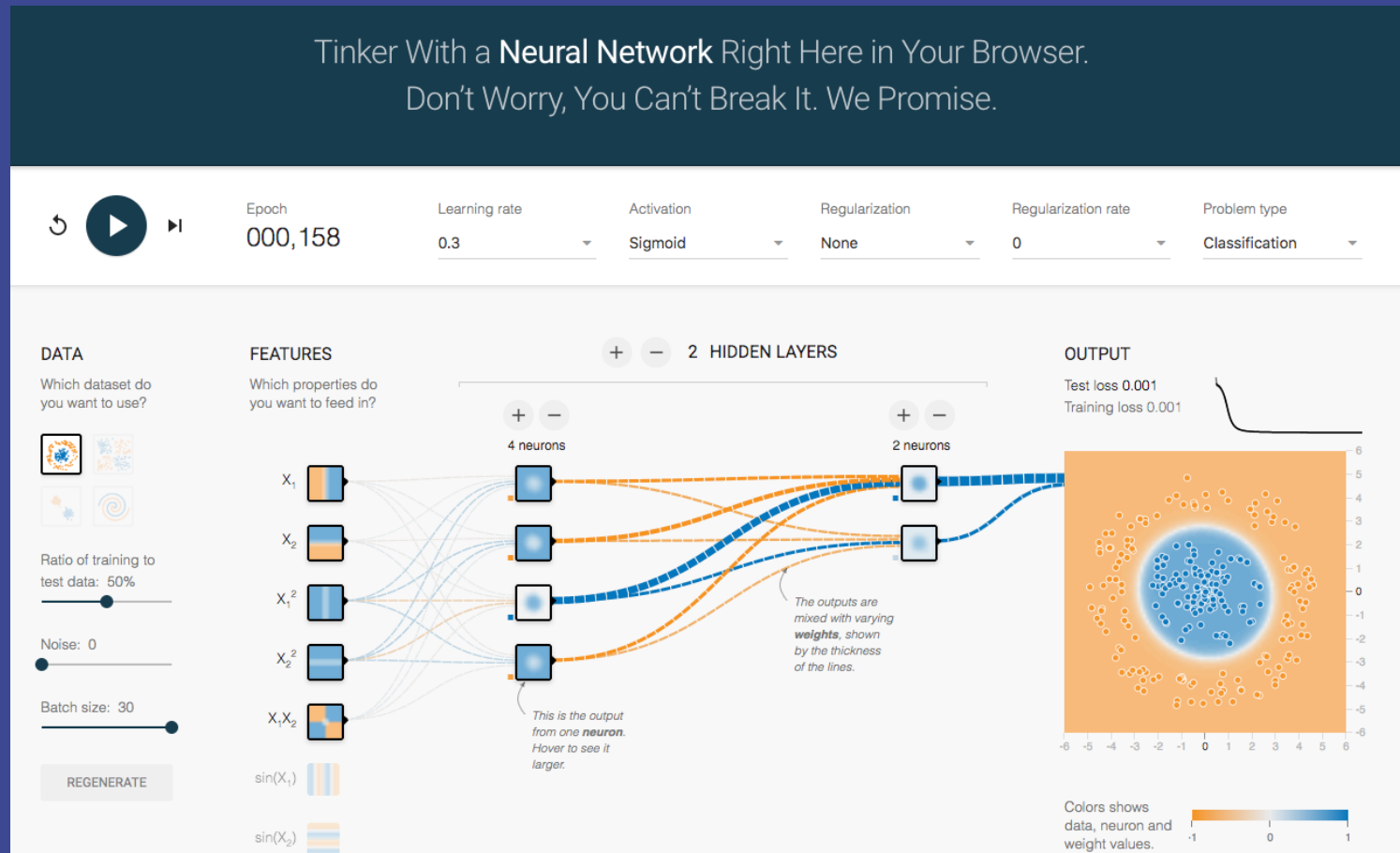


Neural Network Properties

- ▶ Theorem (Universal Function Approximators): A two-layer neural network with a sufficient number of neurons can approximate any continuous function to any desired accuracy.
- ▶ Practical Considerations
 - Can be seen as learning the features
 - Large number of neurons: danger for overfitting
 - Hill-climbing procedure can get stuck in bad local optima

More Neural Nets

- <http://playground.tensorflow.org/>



Case-Based Learning

- Reasoning from examples as opposed to building a model



Nearest Neighbor



Case-Based Reasoning

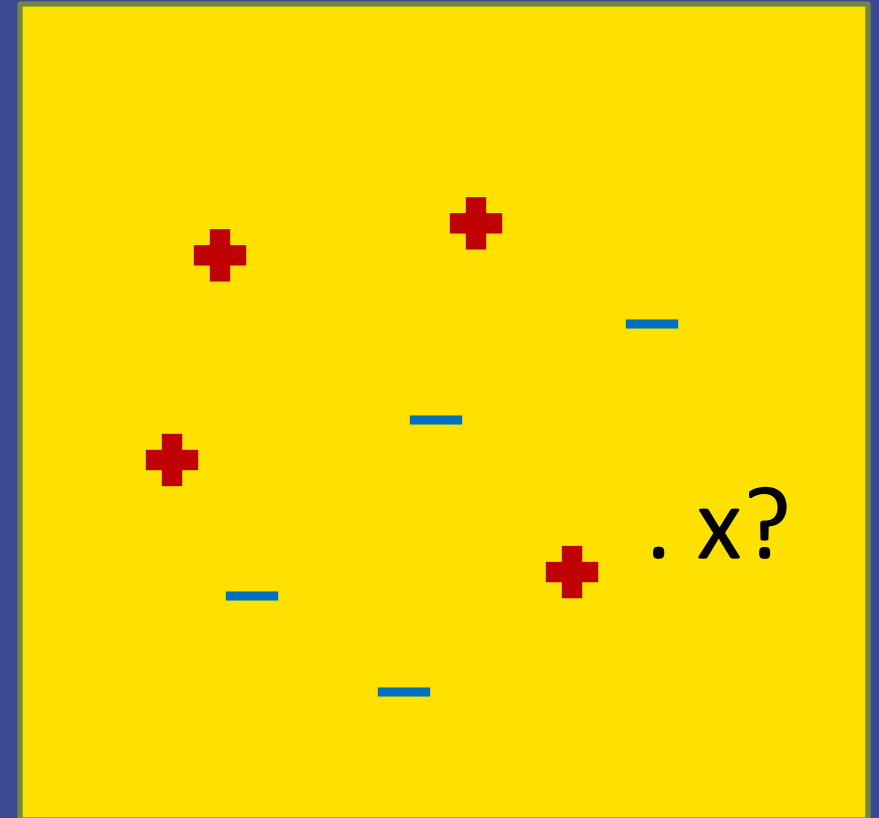
- ▶ Classification from similarity
 - Predict an instance's label using similar instances

Nearest Neighbor Classification

- ▶ 1-NN: copy the label of the **closest** data point in the feature space
- ▶ Closest: most similar

iClicker: What would the label of X be, using 1-NN?

- A. +
- B. -

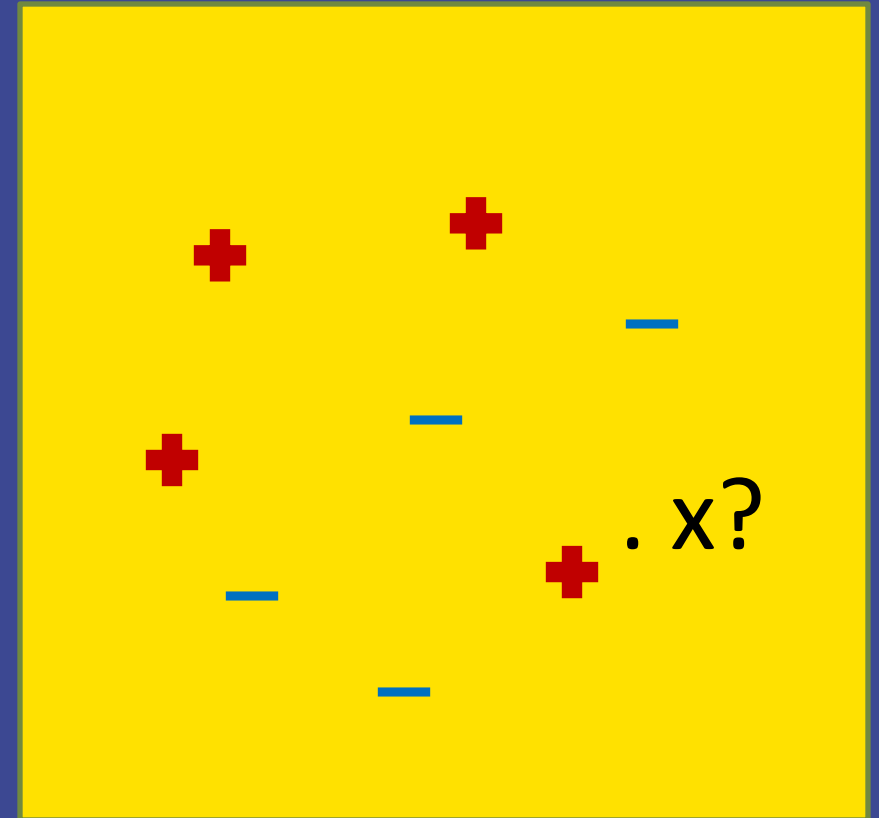


Nearest Neighbor Classification

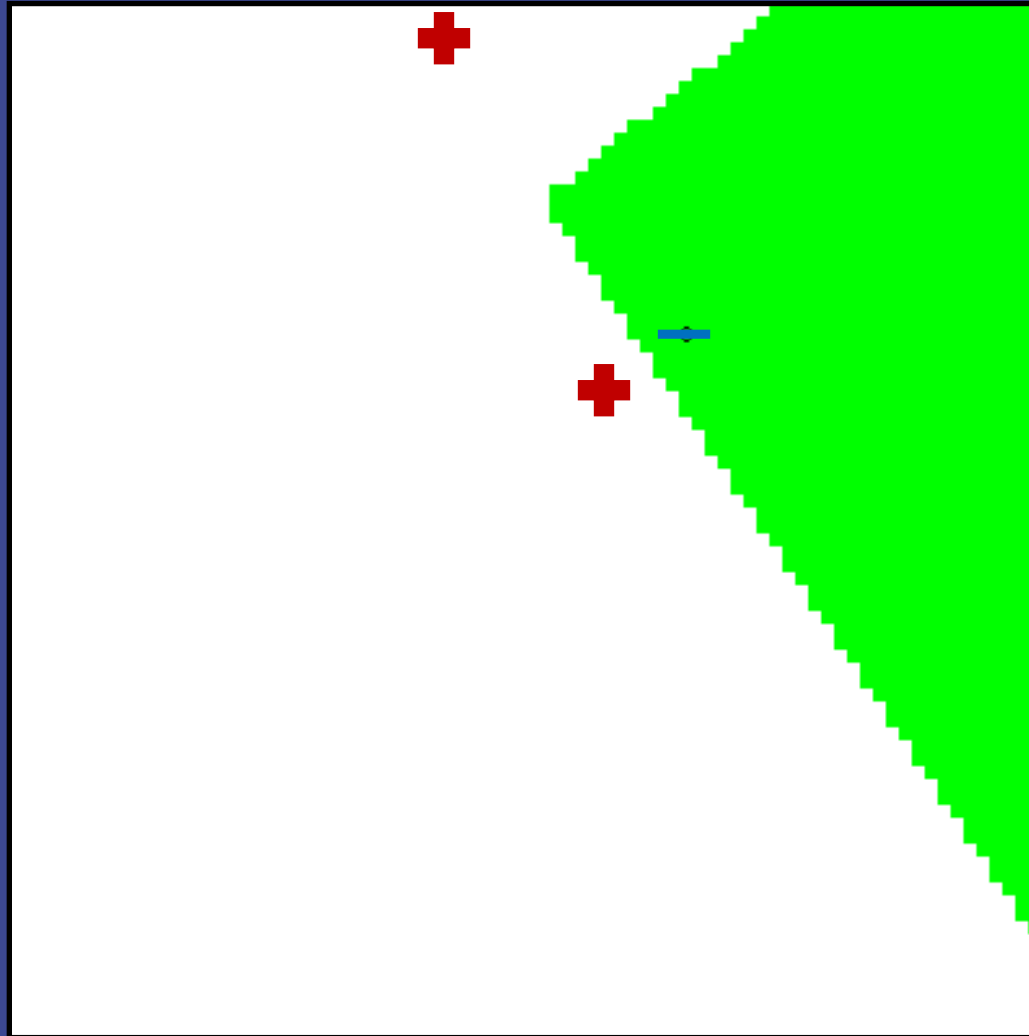
- ▶ **K**-NN: majority vote of the **k** nearest neighbors
- ▶ Large **k** results in **smoother** classification

iClicker: What would the label of X be, using 3-NN?

- A. +
- B. -

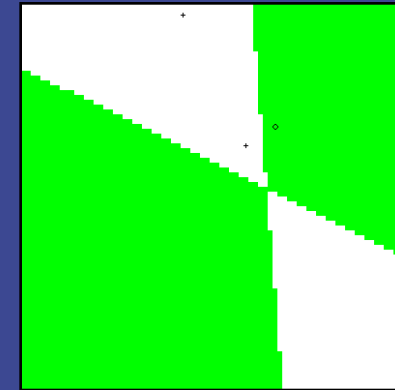


Nearest Neighbor Classification



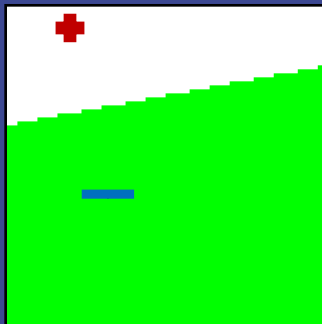
Parametric / Non-Parametric

- ▶ Parametric models:
 - Fixed set of parameters
 - More data means better settings
- ▶ Non-parametric models:
 - Complexity of the classifier increases with data
- ▶ (K)NN is **non-parametric**

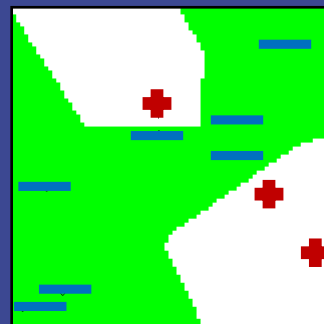


Truth

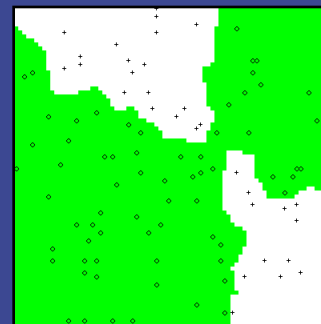
2 Examples



10 Examples



100 Examples



10000 Examples

